

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 11: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper code lives in `starter_code.py` at the week root: the `MaxFlow` class (an Edmonds–Karp maximum-flow solver with `add_edge`, `max_flow`, `flow`, and `min_cut_reachable`), plus `generate_bipartite_graph`, `generate_random_graph`, `edges_to_adj`, `matching_size`, and `is_valid_matching`. Several later problems reuse earlier solutions (e.g. Kuhn’s matching from Problem 2), so the starter files add the `solutions/` directory to the import path.

The 12 problems are grouped into three themes:

- **Bipartite Matching** (Problems 1–6): bipartite testing, maximum matching via both Kuhn’s augmenting-path algorithm and the max-flow reduction, Hall’s deficiency certificate, König’s minimum vertex cover, and assignment feasibility.
- **Flow Applications** (Problems 7–10): edge- and vertex-disjoint paths, baseball elimination, and project selection via min-cut.
- **Reductions** (Problems 11–12): circulation-with-demands feasibility, and verifying the matching/vertex-cover duality (weak in general, tight in bipartite graphs).

1 Bipartite Matching

Problem 1 – Bipartite Check via 2-Coloring

Implement `bipartite_partition(vertices, edges)` that decides whether an undirected graph is bipartite using a BFS 2-coloring. If it is bipartite, return a tuple of two sets (A, B) giving the bipartition; if the graph contains an odd cycle, return `None`. Run BFS from each unvisited vertex, coloring each neighbour the opposite color; if an edge ever joins two same-colored vertices, the graph is not bipartite.

Example. A 4-cycle 0–1–2–3–0 is bipartite ($\{0,2\}$, $\{1,3\}$); a triangle is not.

Problem 2 – Maximum Bipartite Matching via Augmenting Paths (Kuhn’s)

Implement `kuhn_matching(left, right, edges)`: maximum bipartite matching by Kuhn’s augmenting-path algorithm. Return a dict mapping each matched *right* vertex to its *left* partner. For each left vertex, run a DFS that looks for an augmenting path, bumping already-matched right vertices to alternative partners. Also implement `max_matching_size(left, right, edges)`.

Problem 3 – Maximum Bipartite Matching via Max-Flow Reduction

Implement `flow_matching(left, right, edges)` using the `MaxFlow` helper: source $\rightarrow$ each left vertex (capacity 1), each edge left $\rightarrow$ right (capacity 1), each right vertex $\rightarrow$ sink (capacity 1). The saturated left $\rightarrow$ right edges form a maximum matching; return them. Also implement `flow_matching_size`. The tests **verify the flow-based size equals Kuhn’s result** (Problem 2) on random bipartite graphs – two very different algorithms computing the same quantity.

Problem 4 – Hall’s Condition Checker

Implement `hall_violator(left, right, edges)`: decide whether the *left* side has a matching saturating *all* of its vertices. Return `None` if such a matching exists; otherwise return a *deficient* subset S of left vertices with $|N(S)| < |S|$, witnessing the failure of Hall’s condition. Recover S from the alternating-path structure of a maximum matching (Problem 2). Also implement `neighbors_of(subset, adj)`.

Hint. If not all left vertices are matched, the set of left vertices reachable by alternating paths from the unmatched left vertices is deficient.

Problem 5 – König’s Theorem: Min Vertex Cover from Max Matching

Implement `min_vertex_cover(left, right, edges)` via the constructive proof of König’s theorem: find a maximum matching; from the unmatched left vertices, mark all vertices reachable by alternating paths; the cover is (unmarked left) $\cup$ (marked right). Also implement `is_vertex_cover(edges, cover)`. The tests **verify $|\text{cover}| = |\text{matching}|$** on random bipartite graphs.

Problem 6 – Assignment Feasibility (Perfect Matching of One Side)

Implement `can_assign_everyone(people, tasks, allowed)`: return `True` iff every person can be assigned a distinct allowed task (a matching saturating all people). Also implement `assignment(people, tasks, allowed)` returning a `person  $\rightarrow$  task` dict if everyone can be assigned, else `{}`. Build on `kuhn_matching` (Problem 2).

Example. Three people each with at least one distinct allowed task can all be assigned; two people who both only accept a single task cannot.

2 Flow Applications

Problem 7 – Maximum Edge-Disjoint s - t Paths

Implement `max_edge_disjoint_paths(vertices, directed_edges, s, t)`: the maximum number of edge-disjoint directed s - t paths. By Menger's theorem this equals the max flow when every edge has capacity 1 – build that network with `MaxFlow` and return the flow value.

Example. In a graph with two parallel routes $s \rightarrow a \rightarrow t$ and $s \rightarrow b \rightarrow t$, the answer is 2; a single bottleneck edge shared by all routes forces the answer down to 1.

Problem 8 – Vertex-Disjoint s - t Paths via Node Splitting

Implement `max_vertex_disjoint_paths(vertices, directed_edges, s, t)`: the maximum number of internally vertex-disjoint directed s - t paths. Use node splitting: each internal vertex v becomes $v_{\text{in}} \rightarrow v_{\text{out}}$ with capacity 1, and each original edge (u, v) becomes $u_{\text{out}} \rightarrow v_{\text{in}}$ with capacity 1; keep s and t uncapped.

Hint. Compare with Problem 7: two paths can be edge-disjoint yet share an internal vertex (so vertex-disjoint counts can be strictly smaller).

Problem 9 – Baseball Elimination via Max Flow

Implement `is_eliminated(teams, wins, remaining, games_between, x)`: decide whether team x can still possibly finish in first place. Let $W = \text{wins}[x] + \text{remaining}[x]$. If any other team already has more wins than W , x is trivially eliminated. Otherwise build the network: source $\rightarrow$ game node $\{i, j\}$ (capacity = games left between i, j); game node $\rightarrow$ each of i, j (capacity ∞); team $i \rightarrow$ sink (capacity $W - \text{wins}[i]$). Team x survives iff the max flow saturates all game edges.

Note. `games_between` is keyed by unordered pairs given as `tuple(sorted((i, j), key=str))`.

Problem 10 – Project Selection / Max-Weight Closure via Min-Cut

Implement `select_projects(profits, prerequisites)`: choose a prerequisite-closed subset of projects maximizing total profit (profits may be negative). Reduction: source $\rightarrow v$ (capacity `profit[v]`) for profitable v ; $v \rightarrow$ sink (capacity `-profit[v]`) for unprofitable v ; prerequisite edge $p \rightarrow q$ with capacity ∞ . Then $\text{max profit} = (\text{sum of positive profits}) - (\text{min cut})$, and the selected set is the source side (`MaxFlow.min_cut_reachable`). Return `(max_profit, selected_set)`. Also implement `brute_force_select` (try all closed subsets) so the tests can **verify min-cut against brute force** on small inputs.

3 Reductions

Problem 11 – Circulation with Demands: Feasibility Check

Implement `circulation_feasible(vertices, demands, edges)`: decide whether a feasible circulation with demands and lower bounds exists. Each vertex v has a demand d_v (> 0 absorb, < 0 supply); each edge $(u, v, \text{lower}, \text{upper})$ must carry flow in $[\text{lower}, \text{upper}]$. Reduce to max flow: (1) eliminate lower bounds by setting capacity to $\text{upper} - \text{lower}$ and adjusting $d_u += \text{lower}$, $d_v -= \text{lower}$; (2) add a super-source/super-sink for the adjusted supplies/demands; (3) feasible iff adjusted supply equals adjusted demand *and* the max flow saturates all super-source edges.

Example. A source a (-3) feeding a sink b ($+3$) over an edge of capacity ≥ 3 is feasible; the same with capacity 2, or with a lower bound that forces more flow than b can absorb, is infeasible.

Problem 12 – Matching $\leq$ Vertex Cover, with Equality in Bipartite Graphs

Implement `brute_force_max_matching(vertices, edges)` and `brute_force_min_vertex_cover(vertices, edges)` (both for *any* graph, by brute force), and `verify_matching_cover_relationship(num_trials, seed)` which, on random graphs, checks: (1) maximum matching $\leq$ minimum vertex cover always (weak duality); and (2) in *bipartite* graphs maximum matching = minimum vertex cover (Kőnig), with the constructive cover from Problem 5 attaining the bound.

Example. A triangle (non-bipartite) has maximum matching 1 but minimum vertex cover 2 – a strict inequality, confirming Kőnig’s theorem requires bipartiteness.

4 LeetCode Practice

After completing the problems above, practise this week's flow and matching techniques (bipartite checking, bipartite matching, and graph/DAG modeling) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Is Graph Bipartite?](#) – *Medium* – Bipartite check.
- [Possible Bipartition](#) – *Medium* – Bipartite check.
- [Maximum Number of Accepted Invitations](#) – *Medium* – Bipartite matching.
- [Optimal Account Balancing](#) – *Hard* – Flow / search.
- [Minimum Number of Vertices to Reach All Nodes](#) – *Medium* – Graph application.
- [Course Schedule II](#) – *Medium* – DAG application.